

System & Game-System Manager Redesign

Status: design discussion, not yet implemented. Nothing in this document has been written to `src/` — the code sketches below are illustrative, for you to type in by hand. This doc is a working notebook to read through and push back on, not a finished spec.

Executive summary

`Application` currently owns systems ad hoc: a raw `vector<shared_ptr<ISystem>>` plus a separately-typed member (`inputSystem`) kept around *only* so its concrete type is still reachable after being type-erased into that vector. `EntityManager` is scene-owned and manually ticked, with a comment already flagging it as needing to move up to `Application` . Apple/snake respawn logic is either dead code or commented out, driven by direct function calls rather than the event bus.

The redesign introduces one new engine type (`SystemManager`) and one new game-layer type (`GameSystemManager`), both holding their contents by `unique_ptr` and both **living for the whole application, never per-scene**. That last point was the resolving realization of the conversation this document captures: a manager that's recreated per scene can't hold any system that needs to survive a scene transition (a score system carrying state from gameplay into a game-over screen, for instance). So scoping happens one level down — scenes add and remove their *own entities and game systems* through `AddEntity / RemoveEntity` and `AddSystem / RemoveSystem` , using manager references that outlive any single scene.

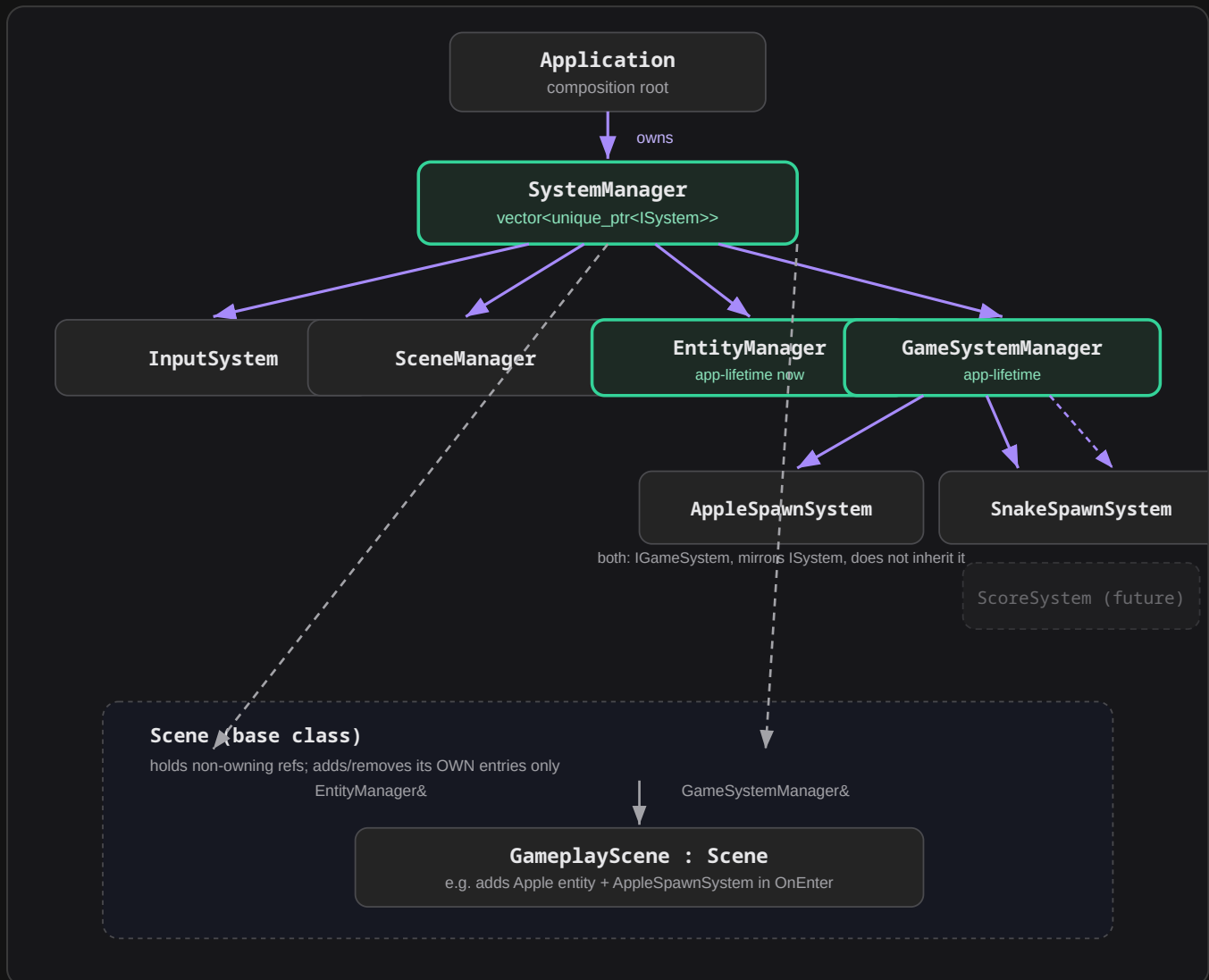
A new `Scene` base class is proposed so concrete scenes (`GameplayScene` and future scenes) inherit the entity/game-system wiring instead of re-declaring it each time.

1. Current state — what prompted this

Symptom	Where	Why it matters
<code>inputSystem</code> stored twice: typed member and in <code>systems</code>	<code>application.hpp:109</code> , <code>application.cpp:74,82</code>	Only reason: needing the concrete type later (<code>GetActionRouter()</code> , <code>application.cpp:279</code>). Not real shared ownership — <code>shared_ptr</code> is doing work <code>unique_ptr</code> + a typed getter could do instead.
<code>EntityManager</code> constructed fresh inside <code>GameplayScene</code> 's own constructor	<code>gameplay-scene.cpp:20-22</code>	Scene-owned, so a second scene visit means a second, distinct <code>EntityManager</code> — no way to persist any entity across a scene boundary.
<code>EntityManager</code> ticked manually by the scene	<code>gameplay-scene.cpp:51,80,84</code>	Flagged in-code: <code>// SIDE QUEST: The Entity Manager function call should be happening with the other Systems in Application .</code>
<code>GameplayScene::OnExit()</code> is empty	<code>gameplay-scene.cpp:73</code>	Nothing is torn down today — not entities, not anything. Any scoping design has to introduce real teardown, not assume it already half-exists.
Apple respawn position-set is commented out	<code>gameplay-state.cpp</code> (collision block), <code>apple.cpp:42-43</code>	<code>// MAIN QUEST: This should be handled by the apple spawner</code> — spawner exists (<code>SnakeGame::AppleSpawner</code>) but has zero call sites anywhere in <code>src/</code> .
Apple-collected handling is direct calls, not events	<code>gameplay-state.cpp:64-75</code>	<code>gameContext.score += 10;</code> <code>snake->SetGrow(true);</code> — no <code>AppleCollectedEvent</code> is published; the stub event file is empty.
<code>ISystem</code> now includes <code>IOOnRegistration</code>	<code>core/systems/i-system.hpp</code>	Already applied — <code>ISystem</code> : <code>IOOnUpdatable</code> , <code>IOOnDebugable</code> , <code>IOOnRegistration</code> . This document builds on that as a given.
Several classes flagged as candidates for <code>ISystem</code>	<code>SceneManager</code> (<code>// RAID BOSS</code>), <code>EntityComponentPipeline</code> (<code>// RAID BOSS</code>), <code>Debug::DebugSystem</code> (<code>// RAID BOSS</code>), plus unflagged partial fits: <code>ActionRouter</code> , <code>InputDispatcher</code> ,	Out of scope for this document, but the <code>SystemManager</code> shape below is exactly what they'd eventually register into.

Symptom	Where	Why it matters
	ComponentDispatcher , GameplayStateMachine	

2. Target shape



■ solid purple = owns (`unique_ptr`) ■ dashed grey = non-owning reference ■ green outline = promoted to app-lifetime by this redesign

Everything in the green-outlined boxes is new or newly-promoted. `InputSystem` and `SceneManager` are drawn for continuity — they don't change here.

3. SystemManager

Sole owner, flat list, no dispatch logic — see §7 for why a dispatcher (à la `ComponentDispatcher` / `InputDispatcher`) doesn't belong here.

```
namespace Engine::Systems {

class SystemManager {
    std::vector<std::unique_ptr<Core::Systems::ISystem>> systems;

public:
    Core::Systems::ISystem* AddSystem(std::unique_ptr<Core::Systems::ISystem> system) {
        system->OnRegistration();
        systems.push_back(std::move(system));
        return systems.back().get();
    }

    void RemoveSystem(Core::Systems::ISystem* system) {
        std::erase_if(systems, [&](auto& s) { return s.get() == system; });
    }

    template <typename T>
    T* GetSystem() {
        for (auto& system : systems) {
            if (auto* match = dynamic_cast<T*>(system.get())) return match;
        }
        return nullptr;
    }

    void OnUpdate(float dt) { for (auto& s : systems) s->OnUpdate(dt); }
    void OnDebugUpdate() const { for (auto& s : systems) s->OnDebugUpdate(); }
    void OnDebugRender() const { for (auto& s : systems) s->OnDebugRender(); }
};

} // namespace Engine::Systems
```

`Application` holds one `SystemManager` instead of `vector<shared_ptr<ISystem>>` (`application.hpp:111`), and drops the standalone `inputSystem` member (`application.hpp:109`) entirely. Wherever the concrete type is needed later (`application.cpp:279`, `GetActionRouter()`), fetch it:

```
this->systemManager.GetSystem<Engine::Input::InputSystem>()->GetActionRouter()
```

`GetSystem<T>()` mirrors `Core::Entities::IEntity::GetComponent<T>()` (`i-entity.hpp:44-64`) on purpose — same shape, same call-site feel. One deliberate difference: `GetComponent<T>` has a `type_index`-keyed fast path for concrete types, falling back to a `dynamic_cast` scan only for abstract `T`. `GetSystem<T>()` skips that fast path — it's a handful of systems, looked up rarely (setup/wiring code), not a per-frame hot path the way component lookups can be. Worth revisiting only if that assumption stops holding.

4. `IGameSystem` — mirrors `ISystem`, does not inherit it

This is the one decision in this document most likely to look like a mistake on a quick read, so it's worth spelling out carefully.

```
namespace Core::Systems {

class IGameSystem {
public:
    virtual ~IGameSystem() = default;
    virtual void OnUpdate(float deltaTime) = 0;
    virtual void OnDebugUpdate() const = 0;
    virtual void OnDebugRender() const = 0;
    virtual void OnRegistration() = 0;
};

} // namespace Core::Systems
```

Same four signatures as `ISystem`. **No** `: public ISystem`.

Why not just inherit? The entire point of having two types is a compile-time guarantee: a game system must not be registerable directly into the top-level `SystemManager`, and an engine system must not be registerable into `GameSystemManager`. If `IGameSystem` publicly inherited `ISystem`, that guarantee evaporates — `unique_ptr<IGameSystem>` converts implicitly to `unique_ptr<ISystem>` (ordinary derived-to-base conversion), so `systemManager.AddSystem(std::move(myGameSystem))` would silently compile. Public inheritance *is* substitutability; substitutability is exactly what's being ruled out here. Mirroring — same shape, unrelated type — is what makes `SystemManager::AddSystem(unique_ptr<ISystem>)` and `GameSystemManager::AddSystem(unique_ptr<IGameSystem>)` genuinely reject each other's types.

The cost is real: four method declarations duplicated, and if `ISystem` grows a fifth hook, someone has to remember to add it to `IGameSystem` too. Given how thin both interfaces are, that seemed like a fair trade for the enforcement — but it's worth naming as a trade rather than a free win.

A nice side effect: because the method *names* match even though the types are unrelated, one free function template works across both managers' `AddSystem`:

```
template <typename T>
T* RegisterInto(std::vector<std::unique_ptr<T>>& container, std::unique_ptr<T> item) {
    item->OnRegistration();
    container.push_back(std::move(item));
    return container.back().get();
}
```

Not extracted in the sketches above — at two call sites (`SystemManager::AddSystem`, `GameSystemManager::AddSystem`) it reads fine written out, and `EntityManager::AddEntity` doesn't fit the same template (it also drives the activation pipeline), so pulling this out now would be templating for a pattern that isn't proven at three real, identical sites yet. Flagging it here so it's not forgotten if a third call site shows up.

5. `GameSystemManager`

```
namespace Engine::Systems {

class GameSystemManager final : public Core::Systems::ISystem {
    std::vector<std::unique_ptr<Core::Systems::IGameSystem>> gameSystems;

public:
    Core::Systems::IGameSystem* AddSystem(std::unique_ptr<Core::Systems::IGameSystem> system) {
        system->OnRegistration();
        gameSystems.push_back(std::move(system));
        return gameSystems.back().get();
    }

    void RemoveSystem(Core::Systems::IGameSystem* system) {
        std::erase_if(gameSystems, [&](auto& s) { return s.get() == system; });
    }

    template <typename T>
    T* GetSystem() {
        for (auto& system : gameSystems) {
            if (auto* match = dynamic_cast<T*>(system.get())) return match;
        }
        return nullptr;
    }

    void OnUpdate(float dt) override { for (auto& s : gameSystems) s->OnUpdate(dt); }
    void OnDebugUpdate() const override { for (auto& s : gameSystems) s->OnDebugUpdate(); }
    void OnDebugRender() const override { for (auto& s : gameSystems) s->OnDebugRender(); }
    void OnRegistration() override { /* currently a no-op – see open questions */ }
};

} // namespace Engine::Systems
```

`GameSystemManager` is itself an `ISystem` — constructed once, registered into `SystemManager` at `Application` bootstrap, ticked automatically from then on. It does **not** hold an `IEventBus` reference. Every individual game system already gets the bus constructor-injected by whoever builds it (the same convention `Application` uses for `InputSystem`, `EntityManager`, etc.), so there's nothing for the manager itself to relay.

6. The resolving insight: containers persist, only entries scope to scenes

This is the point the conversation converged on, and it's easy to get backwards, so it's worth stating as its own rule:

A manager promoted into `SystemManager` lives for the whole application. Scoping to a scene happens by adding/removing *individual entries* inside that manager from that scene's `OnEnter / OnExit` — never by constructing/destroying the manager itself per scene.

The first version of this design (earlier in the conversation, before this realization) had `GameSystemManager` — and by extension `EntityManager` — owned directly by the scene, mirroring how `EntityManager` is scene-owned *today*. That works fine right up until a game system needs to survive a scene transition — a `ScoreSystem` that has to carry a running total from the gameplay scene into a game-over scene, say. A scene-owned manager is destroyed with its scene, taking that state with it. There is no way to have "two versions" of a thing that's supposed to be one continuous thing.

So the fix inverts which layer owns what:

- `EntityManager` and `GameSystemManager` : **one instance each, for the life of the app**, owned by `SystemManager`. Scenes receive references, they don't construct or own these.
- Individual **entities** and individual **game systems**: scoped to whichever scene added them, via ordinary `AddEntity / RemoveEntity` and `AddSystem / RemoveSystem` calls made in that scene's `OnEnter / OnExit`.
- Something that needs to outlive any one scene (the hypothetical `ScoreSystem`) simply never gets `RemoveSystem` 'd by anyone — it stays registered across as many scene transitions as it needs to, and handles "the player started a new game" by subscribing to `GameStartedEvent` and resetting its own internal state, not by being torn down and rebuilt.

This is also why `EntityManager` moving out of `GameplayScene` 's constructor (`gameplay-scene.cpp:20-22`) isn't just tidying up the `// SIDE QUEST` comment — it's load bearing. A scene-owned `EntityManager` cannot support anything that needs entity continuity across scenes, the same way a scene-owned `GameSystemManager` can't support `ScoreSystem`.

7. Why no dispatcher here

`ComponentDispatcher` and `InputDispatcher` solve **type-based routing** — which registrar applies to which component type, which raw input maps to which action. Both have real branching logic keyed off a type or value.

`SystemManager / GameSystemManager` don't have that axis of variation. Every entry gets treated identically: `OnRegistration()` once, `OnUpdate / OnDebugUpdate / OnDebugRender` every frame, for all of them, no per-type dispatch. That's three flat loops over a `vector`, not a dispatch problem — adding a dispatcher layer here would be structure in search of a problem.

8. Game events and the respawn flow

`src/snake-game/game-events/game-started-event.hpp` and `apple-collected-event.hpp` are currently empty files. They get real bodies:

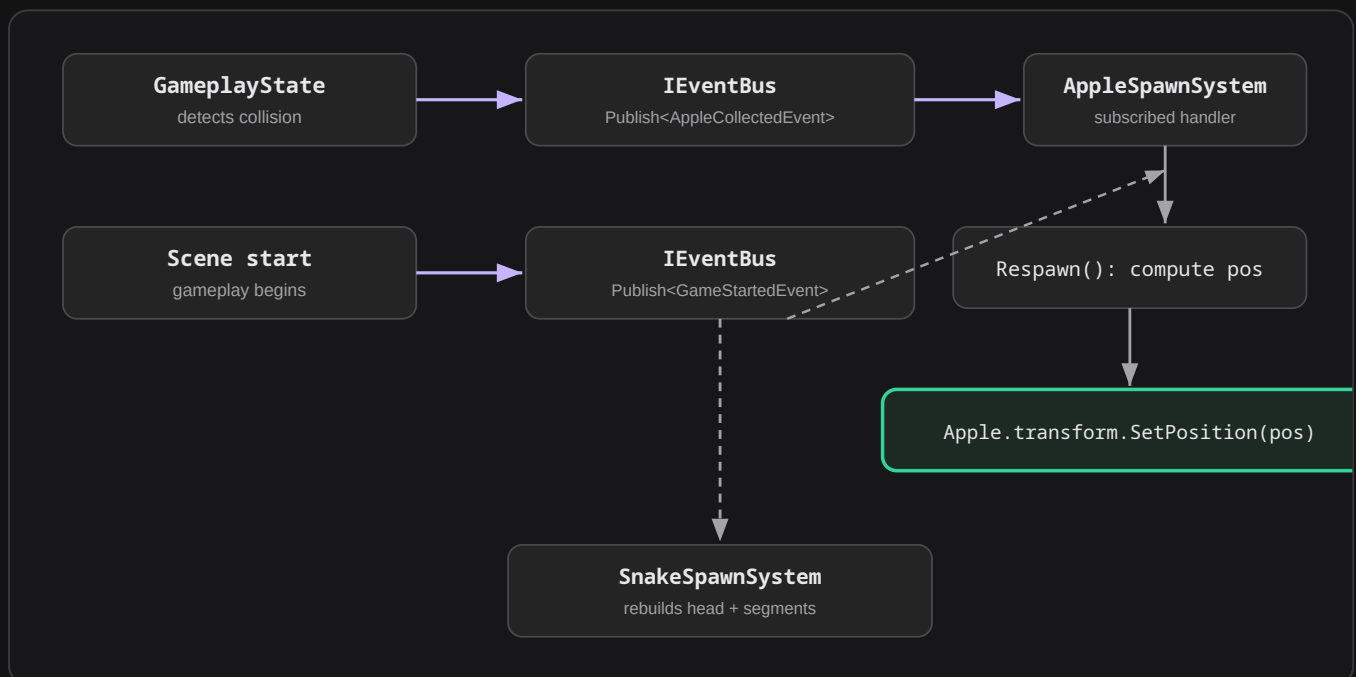
```
// game-started-event.hpp
struct GameStartedEvent : Core::Events::IEvent {};

// apple-collected-event.hpp
struct AppleCollectedEvent : Core::Events::IEvent {};
```

`AppleSpawnSystem` (an `IGameSystem`, replacing the currently-dead, stateless `SnakeGame::AppleSpawner` helper — zero call sites anywhere in `src/` today) subscribes to both in `OnRegistration()`:

```
void AppleSpawnSystem::OnRegistration() {
    eventBus.Subscribe<GameStartedEvent>([this](const auto&) { Respawn(); });
    eventBus.Subscribe<AppleCollectedEvent>([this](const auto&) { Respawn(); });
}
```

`SnakeSpawnSystem` subscribes only to `GameStartedEvent`.



■ solid = direct call on this path ■ dashed = "also triggers" (`GameStartedEvent` fans out to both spawn systems)

The apple-collected collision block in `GameplayState::Update` (`gameplay-state.cpp:64-75`) changes from:

```
gameContext.score += 10;
snake->SetGrow(true);
```


to additionally publishing:

```
eventBus.Publish<AppleCollectedEvent>({});
```

Score/grow can stay direct calls for now, or become their own event later (there's already an empty `score-changed-event.hpp` stub) — not required for this redesign to work.

9. Proposed: `Engine::Scenes::Scene` base class

Not written to `src/`. This section is a design for you to type in — per this repo's read-only-for-Claude policy, I haven't (and won't) create or edit any `.hpp / .cpp` here.

Why

`GameplayScene` currently hand-rolls its own `EntityManager` construction, manual per-frame ticking, and (once this redesign lands) would otherwise need to hand-roll the add-on-enter/remove-on-exit bookkeeping for every entity and game system it creates. Every future scene would repeat that same wiring. A base class collects it once.

Shape

```
namespace Engine::Scenes {

struct SceneParams {
    Core::Entities::IEntityManager& entityManager;
    Engine::Systems::GameSystemManager& gameSystemManager;
    Core::Events::IEventBus& eventBus;
    // ...other shared, app-lifetime references a scene commonly needs
};

class Scene : public Core::Scenes::IScene {
public:
    explicit Scene(const SceneParams& params);
    ~Scene() override = default;

    // Template-method: derived scenes override OnSceneEnter/OnSceneExit, not
    // OnEnter/OnExit directly – see "open question" below on whether this
    // should be enforced (non-virtual OnExit) or left as a convention.
    void OnExit() final;

protected:
    Core::Entities::IEntity* AddEntity(std::unique_ptr<Core::Entities::IEntity> entity);
    void RemoveEntity(Core::Entities::IEntity* entity);

    Core::Systems::IGameSystem* AddGameSystem(std::unique_ptr<Core::Systems::IGameSystem>
system);
    void RemoveGameSystem(Core::Systems::IGameSystem* system);

    Core::Events::IEventBus& GetEventBus() const { return eventBus; }

    // Derived scenes implement scene-specific enter/exit logic here instead
    // of overriding OnEnter/OnExit from IScene directly.
    virtual void OnSceneExit() {}

private:
    Core::Entities::IEntityManager& entityManager;
    Engine::Systems::GameSystemManager& gameSystemManager;
    Core::Events::IEventBus& eventBus;

    // Tracks only what THIS scene added, so OnExit can clean up exactly that
    // – and nothing belonging to another scene or to something app-lifetime.
    std::vector<Core::Entities::IEntity*> ownedEntities;
    std::vector<Core::Systems::IGameSystem*> ownedGameSystems;
};

} // namespace Engine::Scenes
```

```

Core::Entities::IEntity* Scene::AddEntity(std::unique_ptr<Core::Entities::IEntity> entity) {
    auto* raw = entityManager.AddEntity(std::move(entity));
    ownedEntities.push_back(raw);
    return raw;
}

Core::Systems::IGameSystem* Scene::AddGameSystem(std::unique_ptr<Core::Systems::IGameSystem>
system) {
    auto* raw = gameSystemManager.AddSystem(std::move(system));
    ownedGameSystems.push_back(raw);
    return raw;
}

void Scene::OnExit() {
    for (auto* entity : ownedEntities) entityManager.RemoveEntity(entity);
    for (auto* system : ownedGameSystems) gameSystemManager.RemoveSystem(system);
    ownedEntities.clear();
    ownedGameSystems.clear();
    OnSceneExit();
}

```

`GameplayScene` then becomes:

```

class GameplayScene : public Engine::Scenes::Scene {
public:
    using Scene::Scene;

    void OnEnter(Core::Scenes::SceneTransitionContext ctx) override {
        this->transition = ctx;
        this->snake = std::make_unique<Snake>(snakeParams);
        this->snake->Initialize();

        auto apple = std::make_unique<Apple>(appleParams);
        this->apple = this->AddEntity(std::move(apple));
        this->appleSpawnSystem = this->AddGameSystem(std::make_unique<AppleSpawnSystem>(...));
    }

    // No Update()/DebugUpdate()/DebugRender() forwarding to entityManager anymore –
    // SystemManager ticks EntityManager and GameSystemManager automatically now
    // that both are app-lifetime, registered systems.

private:
    Core::Entities::IEntity* apple = nullptr;
    Core::Systems::IGameSystem* appleSpawnSystem = nullptr;
    std::unique_ptr<Snake> snake;
};

```

Note what disappears from `GameplayScene`: its own `EntityManager` member (`gameplay-scene.hpp:57`), the manual `entityManager->OnUpdate(deltaTime)` / `OnDebugUpdate()` / `OnDebugRender()` forwarding (`gameplay-scene.cpp:51,80,84`), and the construction call in its constructor (`gameplay-scene.cpp:20-22`). All of that lives once, in `Scene`, now.

Open questions (not settled — pick these when you sit down to write it)

- `OnExit` as `final` + a separate virtual `OnSceneExit()` hook, vs. just leaving `OnExit` virtual and documenting "call the base implementation first." The template-method version (shown above) makes it impossible for a derived scene to forget the cleanup step. The plain-virtual version is less machinery but relies on discipline — the same tradeoff you weighed for `IGameSystem` (mirror for a guarantee) but in the other direction (here, enforcement costs a second hook name instead of a second interface).
- Does `GameSystemManager::OnRegistration()` truly stay a no-op forever? Right now nothing gives the manager a reason to subscribe to anything itself. If that changes (e.g. it needs to react to something at the point it's registered, rather than relaying to children), this needs revisiting — noted in §5.
- Is `RegisterInto<T>()` (§4) worth extracting now, or only once a third real call site shows up? Leaned toward "wait" in this document; flagging so it isn't silently forgotten.
- Should `SceneParams` grow to include everything a scene might need (render manager, input router, settings), or should `Scene` only carry `entityManager` / `gameSystemManager` / `eventBus` and let concrete scenes keep taking the rest through their own params struct (as `GameplaySceneParams` does today)? The sketch above assumes the latter — `Scene` owns only the wiring this document is about, not scene construction in general.

10. Summary

Problem	Resolved by
<code>Application</code> dual-stores <code>inputSystem</code> as both a typed member and a type-erased entry	<code>SystemManager::GetSystem<T>()</code> — fetch the concrete type on demand, don't keep a second handle
No compile-time separation between "engine system" and "game system"	<code>IGameSystem</code> mirrors <code>ISystem</code> 's shape without inheriting it
<code>SystemManager</code> / <code>GameSystemManager</code> don't need per-type routing	No dispatcher — three flat loops is the right amount of structure
A scene-owned manager can't hold state that survives a scene transition	Promote <code>EntityManager</code> / <code>GameSystemManager</code> to app-lifetime, <code>SystemManager</code> -owned singletons; scope at the entity/game-system level instead
Apple/snake respawn is dead code or commented out	<code>AppleSpawnSystem</code> / <code>SnakeSpawnSystem</code> , event-driven via <code>GameStartedEvent</code> / <code>AppleCollectedEvent</code>
Every new scene re-implements entity/game-system wiring	Proposed <code>Engine::Scenes::Scene</code> base class (design only — not written)

11. Suggested adoption order

1. Add `SystemManager` to `Engine`, wire `Application` to use it in place of `vector<shared_ptr<ISystem>>`; drop the `inputSystem` dual-storage.
2. Add `IGameSystem` + `GameSystemManager`; register the (currently empty) `GameSystemManager` into `SystemManager` at bootstrap.
3. Promote `EntityManager` out of `GameplayScene`'s constructor into `Application` bootstrap, registered the same way.
4. Fill in `GameStartedEvent` / `AppleCollectedEvent`; write `AppleSpawnSystem` / `SnakeSpawnSystem`; publish the events from their real trigger points.
5. Write the `Scene` base class; migrate `GameplayScene` to inherit from it, deleting the manual entity-manager wiring it currently does itself.

Steps 1–3 are pure plumbing and independently testable (build should stay green after each). Step 4 is where respawn behavior actually starts working. Step 5 is cleanup that only pays off once there's a second scene to prove the base class actually saves repetition — worth doing, but not blocking anything above it.